

6.9 Sorting lists

sort()

One of the most useful list methods is `sort()`, which performs an in-place sort of the list elements, sorting the elements from lowest to highest.

PARTICIPATION ACTIVITY

6.9.1: Sorting a list using `list.sort()`.

Start

2x speed

```
my_list = [ 150, 47, 500, -37, 0]
my_list.sort()
```

```
my_list
[ -37 , 0,  47, 150
```

Captions ^

1. The list `my_list` is created and holds integer values.
2. The list is sorted in-place.

Figure 6.9.1: list.sort() method example: Alphabetically sorting book titles.

```
books = []
prompt = "Enter new book: "
user_input = input(prompt).strip()

while (user_input.lower() != "exit"):
    books.append(user_input)
    user_input = input(prompt).strip()

books.sort()

print("\nAlphabetical order:")
for book in books:
    print(book)
```

```
Enter new book: Pride, Pre
Enter new book: Programmi
Enter new book: Hackers ar
Enter new book: World War
Enter new book: exit
```

```
Alphabetical order:
Hackers and Painters
Pride, Prejudice, and Zomk
Programming in Python
World War Z
```

sorted()

The sorted() built-in function provides the same sorting functional method, however, sorted() creates and returns a new list instead of

Figure 6.9.2: Using sorted() to create a new sorted list from an existing list with

```
numbers = [int(i) for i in input("Enter numbers: ").split()]

sorted_numbers = sorted(numbers)

print(f"\nOriginal numbers: {numbers}")
print(f"Sorted numbers: {sorted_numbers}")
```

```
Enter numbers: 1 2 3 4 5
Original numbers: [1, 2, 3, 4, 5]
Sorted numbers: [1, 2, 3, 4, 5]
```

key

Both the list.sort() method and the built-in sorted() function have a key parameter. The key specifies a function to be applied to each element prior to

Figure 6.9.3: Using the key argument.

```

names = []
prompt = "Enter name: "

user_input = input(prompt)

while user_input != "exit":
    names.append(user_input)
    user_input = input(prompt)

no_key_sort = sorted(names)
key_sort = sorted(names, key=str.lower)

print(f"Sorting without key: {no_key_sort}")
print(f"Sorting with key: {key_sort}")

```

```

Enter name: Serena Williams
Enter name: Venus Williams
Enter name: rafaël Nadal
Enter name: john McEnroe
Enter name: exit
Sorting without key: ['Serena Williams', 'Venus Williams', 'john McEnroe']
Sorting with key: ['john McEnroe', 'rafael Nadal', 'Serena Williams']

```

Figure 6.9.4: The key argument to list.sort() or sorted() can be assigned any function.

```

my_list = [[25], [15, 25, 35], [10, 15]]

sorted_list = sorted(my_list, key=max)

print(f"Sorted list: {sorted_list}")

```

```
Sorted list: [[10, 15],
```

reverse

Sorting also supports the reverse argument. The reverse argument value, either True or False.